TOP 1% AI PRODUCT ENGINEER

COMPLETE MASTER ROADMAP — v2.0

Frontend Developer → Production AI Systems Engineer

Target: ₹30L–₹1Cr+ | Timeline: 4–6 Years | 8 Phases | 7 Portfolio Projects

Profile	Frontend Developer — 2 years — JS, React, HTML, CSS
Target Role	AI Product Engineer — Design, Build, Deploy & Scale AI SaaS at production
Salary Target	₹30L+ in 4–6 years; ₹60L–₹1Cr+ or $100K–$150K remote by Year 6
Document v2	Fixes: AI infra depth, data engineering, modern deploy stack, AI security, LLM ops
Total Phases	8 phases covering Frontend → Backend → Python → ML → LLMs → AI Infra → Data Eng → MLOps/Safety
Portfolio	7 production-grade GitHub projects designed to get top-tier job offers
Philosophy	Build first, theorise second. Ship broken things weekly. Learn in public. Your GitHub is your CV.

🗺 Roadmap Overview & Core Philosophy

This is a practical engineering curriculum, not an academic syllabus. Every week you build something. Every month you ship something. The goal is not to know everything about AI — it is to be able to design, build, deploy, and scale AI-powered products that users pay for. You already have a valuable asset: frontend skills. The strategy is to add layers on top, not start over.

8-Phase Architecture

Phase 1 (Mo 1–2)	Advanced JS + React + Next.js — production frontend mastery
Phase 2 (Mo 3–5)	Backend Engineering — Node.js, APIs, PostgreSQL, Redis, Auth, System Design
Phase 3 (Mo 6–8)	Python for AI — Python, NumPy, Pandas, FastAPI, data pipelines
Phase 4 (Mo 9–13)	ML + LLM Engineering — transformers, LLMs, embeddings, RAG, agents, fine-tuning
Phase 5 (Mo 14–17)	AI Application Engineering — LangChain, LlamaIndex, LangGraph, AI SaaS architecture
Phase 6 (Mo 18–20)	AI Infrastructure — vLLM, Ray Serve, local LLMs, GPU inference, model serving
Phase 7 (Mo 21–22)	Data Engineering — ETL, Airflow, feature stores, data versioning, quality monitoring
Phase 8 (Mo 23–24)	MLOps + AI Safety — MLflow, W&B, evals, guardrails, prompt injection defense

⚡ **CRITICAL RULE:** Learn a topic for 2 weeks maximum, then BUILD something with it. The project teaches more than the tutorial. If you haven't shipped something in 14 days, you are in tutorial hell. Stop watching. Start building.

PHASE 1 — Advanced JavaScript, React & Next.js Mastery *— Month 1–2*

You already know JS and React. This phase closes every gap and elevates you to production-grade frontend engineer capable of building slick AI UIs with streaming, real-time updates, type safety, and performance. Do not skip this — your frontend skills are your biggest competitive advantage as an AI engineer.

Skill / Technology	Label	YouTube Resource	Alt Resource	Practical Project	Stop Condition	Notes
JavaScript Engine, Closures, Prototypes	JS Core	Akshay Saini — Namaste JS (YouTube, free)	javascript.info (complete free book)	Build a promise library + event emitter from scratch	Explain event loop, microtask queue, closure, prototype chain without notes	Deep JS = ability to debug AI streaming issues, memory leaks in long-running apps
Advanced Async — Promises, Generators, Streams	Async JS	Akshay Saini — Season 2 (YouTube)	You Don't Know JS: Async (GitHub, free)	Async rate-limiter + concurrent queue with backpressure	Build Promise.all, Promise.race from scratch; handle ReadableStream correctly	LLM responses use ReadableStream — you must know streams cold
TypeScript — Strict Mode & Advanced Types	TypeScript	Matt Pocock — Total TypeScript (YouTube)	TypeScript official docs + type-challenges GitHub	Rewrite your AI Chat UI with strict TS — zero 'any' allowed	Write custom utility types, discriminated unions, infer keyword without docs	Non-negotiable for production AI SaaS — all top companies require strict TS
React Internals — Fiber, Reconciler, Concurrent	React Core	Jack Herrington — React 18 Deep Dives (YouTube)	React.dev beta docs (official)	Implement usePrevious, useLatest, useDebounce hooks from scratch	Can explain Fiber, batching, transitions, Suspense boundaries verbally	Concurrent features are critical for smooth AI streaming UIs
State Management — Zustand + TanStack Query	State Mgmt	Theo (t3.gg) — state management deep dives	Zustand docs + TanStack Query v5 docs	Build a multi-tab AI conversation manager with optimistic updates	Zero prop drilling; server state never in useState; offline-capable	TanStack Query for server/LLM data, Zustand for UI state — the modern combo
Next.js App Router, RSC, Server Actions	Next.js	Vercel official YouTube + Next.js conf talks	Next.js 14 docs + Josh W Comeau blog	Full-stack AI app: Next.js + streaming Server Actions + edge runtime	Ship an app to Vercel with ISR + streaming with Suspense in <1 day	Most AI SaaS companies use Next.js — know it deeply
Frontend Performance — Core Web Vitals	Performance	Theo (t3.gg) — web perf series	web.dev performance guide + Chrome DevTools docs	Audit a React app — achieve LCP<2.5s, CLS<0.1, INP<200ms	Pass all Core Web Vitals green on PageSpeed Insights with an AI app	AI streaming UIs have unique perf challenges — token rendering, re-renders
Testing — Vitest + Testing Library + Playwright	Testing	Kent C Dodds — Testing React (YouTube)	Vitest docs + Playwright docs	80%+ test coverage on AI chat component with streaming mocks	Write tests before fixing bugs; mock streaming responses cleanly	AI output is non-deterministic — solid test infra is

Skill / Technology	Label	YouTube Resource	Alt Resource	Practical Project	Stop Condition	Notes
						essential from day 1

Phase 1 — Capstone Project

PROJECT #1: AI Streaming Chat Application

- Next.js 14 App Router + TypeScript strict mode + Tailwind + Shadcn/UI
- Streaming LLM responses via Vercel AI SDK — token-by-token rendering with Suspense
- Multi-model support: GPT-4o, Claude 3.5 Sonnet, Gemini 1.5 Pro — swappable via env vars
- Conversation history with Zustand — persisted to localStorage then Postgres
- Markdown + code syntax highlighting, copy button, regenerate, edit message
- Full TypeScript strict, 80%+ Vitest coverage, Playwright E2E tests
- Deployed: Vercel (free tier). Live URL in README. This is Portfolio Project #1.
- GitHub: Detailed README, architecture diagram, 'How streaming works' explainer section

PHASE 2 — Backend Engineering *— Month 3–5*

AI products are not just frontends calling OpenAI. They need robust backends: auth, multi-tenancy, rate limiting by plan, usage metering for billing, background job queues for async AI tasks, and reliable data storage. This phase makes you genuinely full-stack.

Skill / Technology	Label	YouTube Resource	Alt Resource	Practical Project	Stop Condition	Notes
Node.js Runtime — Streams, Cluster, Workers	Node Core	Hussein Nasser — Node.js internals (YouTube)	Node.js official docs + 'Node.js Design Patterns' book	Build an HTTP/1.1 server from raw net.Socket — no frameworks	Explain streams, backpressure, cluster module, worker threads verbally	AI processing tasks use streams and workers — know the runtime deeply
REST API Design — Fastify + OpenAPI	REST APIs	Matteo Collina (Fastify creator) talks — YouTube	Fastify docs + OpenAPI 3.1 spec	Build a fully documented REST API with Zod validation + OpenAPI spec	Auto-generate API docs from code; consistent error schema across all endpoints	Prefer Fastify over Express — 2–3x faster; better TypeScript support
PostgreSQL + Prisma ORM	SQL	Lama Dev — PostgreSQL + Prisma (YouTube)	Prisma docs + 'Learning PostgreSQL' book	Design multi-tenant SaaS schema: orgs → users → workspaces → AI sessions	Write complex CTEs, window functions, proper indexes; explain EXPLAIN output	Most AI SaaS apps store user data, conversations, embeddings metadata in Postgres
Redis — Caching, Sessions, Pub/Sub	Redis	Fireship — Redis in 100s + Hussein Nasser Redis series	Redis University (free) + Redis docs	Add semantic cache for LLM responses (exact + fuzzy cache layer)	Cache hit rate >60% on repeated AI queries; pub/sub for real-time events	LLM semantic caching = biggest cost reduction lever. Redis is essential.
Authentication — Auth.js + Clerk + JWT	Auth	Theo (t3.gg) — Auth deep dive (YouTube)	Auth.js v5 docs + Clerk docs	Implement OAuth (Google/GitHub) + magic links + API key auth for B2B	Multi-tenant auth where users belong to orgs with RBAC — no bugs	Use Clerk for speed; know JWT internals for debugging security issues
WebSockets + SSE — Real-time AI Streaming	Realtime	Hussein Nasser — SSE vs WebSockets (YouTube)	MDN SSE docs + socket.io docs	Production SSE endpoint with reconnection, backpressure, heartbeat	Serve 500 concurrent SSE streams without memory leak or CPU spike	SSE is the standard for LLM token streaming — master reconnection logic
Background Jobs — BullMQ + Inngest	Jobs/Queues	Theo — background jobs in Next.js (YouTube)	BullMQ docs + Inngest docs	Async AI pipeline: upload PDF → parse → embed → index (all queued)	Zero dropped jobs; idempotent handlers; dead-letter queue for failures	AI tasks (embedding, indexing, processing) must be async — critical pattern

Skill / Technology	Label	YouTube Resource	Alt Resource	Practical Project	Stop Condition	Notes
API Security — Rate Limiting, Helmet, OWASP	Security	Hussein Nasser — API security (YouTube)	OWASP API Security Top 10 guide	Implement per-user, per-plan rate limits tied to token consumption	Pass OWASP Top 10 self-audit; no sensitive data in logs or responses	Rate limiting protects AI API costs — a DDoS can cost you thousands
System Design Fundamentals	Sys Design	Gaurav Sen — System Design (YouTube, full series)	Alex Xu — System Design Interview Vol 1+2	Design an AI document processing system on paper — discuss tradeoffs	Design any system in 30 min: identify bottlenecks, propose solutions	Start early — Senior AI Engineer roles require strong system design skills
Stripe — Usage-Based Billing for AI SaaS	Billing	Theo — Stripe integration (YouTube)	Stripe docs + Stripe Press engineering blog	Token-based billing: meter OpenAI token usage per user per plan tier	Handle webhooks, failed payments, plan upgrades/downgrades without bugs	Token-based billing is the dominant model for AI SaaS — non-negotiable

Phase 2 — Capstone Project

PROJECT #2: AI SaaS Starter Platform

- Complete full-stack SaaS backend open-sourced as 'ai-saas-starter'
- Stack: Next.js 14 + Fastify + PostgreSQL (Prisma) + Redis + Clerk auth + Stripe
- Multi-tenancy: organisations → members → workspaces → AI usage per workspace
- Usage metering: token consumption tracked per user, per plan tier, per day
- Background jobs: BullMQ queue for async AI tasks (document processing, email)
- Observability: Pino structured logging, Sentry error tracking, PostHog analytics
- AI integration: OpenAI + Anthropic providers behind a unified adapter interface
- Deploy: Railway (backend) + Vercel (frontend). One-command local setup with Docker Compose.
- GitHub: Post on HN 'Show HN' — free SaaS boilerplates get 500+ stars. Portfolio Project #2.

PHASE 3 — Python for AI Engineering *— Month 6–8*

Python is the operating language of AI. You don't need to become a Python wizard — you need to be fluent enough to run ML pipelines, write FastAPI microservices, process data, call AI libraries, and write scripts that integrate into your JS stack. This phase covers exactly that.

Skill / Technology	Label	YouTube Resource	Alt Resource	Practical Project	Stop Condition	Notes
Python Fundamentals + Pythonic Patterns	Python	Tech With Tim — Python full course (YouTube)	python-tutorial.eu + Real Python (realpython.com)	Rewrite a Node.js REST API you already built — in FastAPI	Write Python fluently: comprehensions, decorators, generators, context managers	Focus on practical patterns — don't spend weeks on theory
Virtual Envs, Poetry, pyproject.toml	Env Mgmt	ArjanCodes — Python project setup (YouTube)	Poetry docs + Rye (new fast tool) docs	Set up a Python monorepo with Poetry + pre-commit hooks + Ruff linter	Share project; anyone can set it up with 'poetry install' in 60 seconds	Broken Python envs waste days — nail this before touching ML libraries
NumPy — Tensor Math & Broadcasting	NumPy	Keith Galli — NumPy full course (YouTube)	NumPy official quickstart + Stanford CS231n notes	Implement a neural network forward pass in pure NumPy	Matrix multiplication, broadcasting without loops — no shape errors	NumPy understanding = ability to debug tensor shape errors in ML models
Pandas — Data Cleaning & Transformation	Pandas	Rob Mulla — Pandas tutorial (YouTube)	Kaggle Pandas micro-course (free, 4 hours)	Clean a real messy dataset (Kaggle) — missing values, duplicates, normalization	Clean, join, reshape, aggregate any dataset in <30 minutes	RAG pipelines start with dirty data — Pandas is the first cleanup tool
FastAPI — Async Python APIs	FastAPI	ArjanCodes — FastAPI tutorial (YouTube)	FastAPI official docs (best API docs in existence)	Build a FastAPI service with async endpoints, Pydantic validation, OpenAPI docs	Write async Python APIs as confidently as Express — full test coverage	FastAPI is how ML models get exposed as APIs — used at every AI company
Async Python — asyncio, httpx, aiofiles	Async Py	ArjanCodes — Async Python deep dive (YouTube)	Python asyncio docs + Trio docs	Concurrent document downloader: fetch 1000 URLs with rate limiting	Run 200 async tasks without blocking; understand event loop vs threads	LLM API calls are I/O-bound — async is critical for throughput
File I/O — PDF, DOCX, HTML, Images	File Parsing	Corey Schafer — file handling (YouTube)	PyMuPDF docs + python-docx docs + Beautiful Soup 4	Build a universal document parser: PDF/DOCX/HTML/CSV → clean text + metadata	Parse any document type; extract tables, headings, metadata reliably	Foundation of all RAG document ingestion pipelines — critical skill
Data Pipelines — Prefect (intro)	Data Pipes	Prefect official YouTube channel	Prefect 2.0 docs (best beginner-	Build a data ingestion pipeline: scrape → clean	Pipeline runs reliably on schedule; failures	Introduction to Prefect here —

Skill / Technology	Label	YouTube Resource	Alt Resource	Practical Project	Stop Condition	Notes
			friendly orchestrator)	→ embed → store in vector DB	alert and retry automatically	deeper in Phase 7 Data Engineering

Phase 3 — Capstone Project

PROJECT #3: AI Document Intelligence Pipeline

- Python FastAPI microservice that ingests PDF/DOCX/HTML files and extracts structured data
- Multi-format parser: PDF (PyMuPDF), DOCX (python-docx), HTML (BeautifulSoup4), images (Tesseract OCR)
- AI extraction: OpenAI structured outputs to extract entities, tables, key facts as typed JSON (Pydantic)
- Async pipeline: upload → parse → extract → store → webhook notification (all async with BullMQ-style queues)
- Connected to Phase 2 backend: REST API exposed via FastAPI, consumed by Next.js frontend
- Data versioning: store raw + processed versions; re-process on model upgrade
- Deployed on Railway (FastAPI) — integrates with the SaaS Starter from Phase 2
- GitHub: Portfolio Project #3 — practical AI data engineering tool interviewers love

PHASE 4 — ML Foundations + LLM Engineering + RAG + Agents *— Month 9–13*

This is the core AI engineering phase. You will understand how LLMs work internally, master the full RAG stack from naive to advanced, implement AI agents with LangGraph, and learn fine-tuning. This phase alone separates AI engineers from developers who just call APIs.

Section A — ML Foundations & Transformers

Skill / Technology	Label	YouTube Resource	Alt Resource	Practical Project	Stop Condition	Notes
ML Intuition — Supervised, Unsupervised, Evaluation	ML Core	StatQuest with Josh Starmer (YouTube, full series)	fast.ai — Practical Deep Learning (free course)	Train a real classifier on Kaggle data; evaluate with proper train/val/test split	Explain bias-variance tradeoff, overfitting, evaluation metrics without notes	Know the concepts, not the maths — you are building products, not writing papers
Neural Networks — Forward Pass, Backprop	Neural Nets	3Blue1Brown — Neural Network series (YouTube)	Michael Nielsen — NN book (neuralnetworksanddeeplearning.com)	Implement a 2-layer neural network in NumPy from scratch — train on MNIST	Forward pass + backprop + gradient descent — no black boxes	This is the foundation. Spend 1 week, then move on — don't go deeper.
Transformers & Self-Attention Internals	Transformers	Andrej Karpathy — Let's Build GPT (YouTube, 4 hrs)	The Illustrated Transformer — Jay Alammar blog (free)	Implement a mini GPT following Karpathy's video exactly	Explain multi-head attention, positional encoding, KV cache verbally	THE most important architecture in AI. Watch Karpathy's video twice.
HuggingFace Transformers + Datasets	HuggingFace	HuggingFace official YouTube channel	HuggingFace course (huggingface.co/learn, free)	Fine-tune BERT for text classification on a custom dataset	Load, run inference, fine-tune any HuggingFace model in <1 hour	HuggingFace is the npm of ML — every AI engineer uses this daily

Section B — LLM Engineering

Skill / Technology	Label	YouTube Resource	Alt Resource	Practical Project	Stop Condition	Notes
LLM APIs — OpenAI, Anthropic, Gemini, Mistral	LLM APIs	OpenAI Developer Day talks (YouTube)	OpenAI Cookbook (GitHub) + Anthropic docs	Build a multi-provider LLM router with fallback and cost tracking	Swap LLM providers without changing application code	Abstract the provider — never hard-code to one.

Skill / Technology	Label	YouTube Resource	Alt Resource	Practical Project	Stop Condition	Notes
						Providers go down.
Prompt Engineering — Advanced Techniques	Prompting	Andrew Ng + Isa Fulford — ChatGPT Prompt Engineering (free)	DAIR.AI — Prompt Engineering Guide (GitHub, free)	Build a prompt playground with A/B testing, version control, analytics	Reliably extract structured JSON from any LLM with <2% failure rate	CoT, few-shot, self-consistency, tree-of-thought, metacognition prompts
Structured Outputs + Function Calling	Fn Calling	Fireship — OpenAI function calling (YouTube)	OpenAI structured outputs docs + Instructor library	Build a natural language → SQL query engine with structured outputs	Zero invalid JSON from LLM; schema validated with Pydantic/Zod always	Instructor (Python) and Zod + Vercel AI SDK (JS) — use both
Embeddings — Semantic Search & Distance	Embeddings	Andrej Karpathy — embeddings video (YouTube)	OpenAI embeddings guide + MTEB leaderboard	Semantic search over 10K+ documents — evaluate recall@5 accuracy	Semantic search returns relevant results; understand cosine vs dot product	Choose embedding models with MTEB benchmark — not just OpenAI text-3-large
Vector Databases — Qdrant + Pinecone	VectorDB	Pinecone YouTube channel + Qdrant tutorials	Qdrant docs + Pinecone docs + Weaviate docs	Index 100K+ chunks; implement hybrid search (BM25 + vector) with metadata filter	Retrieval latency <100ms at 100K vectors; understand HNSW index internals	Qdrant for self-hosting, Pinecone for managed — know when to use each
RAG — Naive to Production-Grade	RAG	Sam Witteveen — RAG series (YouTube)	LlamaIndex docs + Anthropic RAG guide	Build RAG chatbot over 500-page document; measure retrieval accuracy	Answer accuracy >85% on domain-specific QA benchmarks you define	Naive → chunking strategies → re-ranking → HyDE → query expansion → RAPTOR
Advanced RAG — Re-ranking, HyDE, RAPTOR	Adv RAG	AI Makerspace — Advanced RAG (YouTube)	Cohere re-ranking docs + LlamaIndex advanced RAG guide	Improve naive RAG recall@5 from 60% to 90% using re-ranking + HyDE	Measurable improvement on eval dataset with before/after benchmark numbers	Re-ranking with cross-encoders (Cohere or local) = biggest RAG quality boost
Fine-tuning — LoRA, QLoRA, PEFT	Fine-tuning	Maxime Labonne — LLM fine-tuning (YouTube + blog)	HuggingFace PEFT docs + Axolotl framework	Fine-tune Llama 3.1 8B on a custom Q&A dataset using QLoRA on Colab	Fine-tuned model beats GPT-4o-mini on your domain task by >20%	Use fine-tuning only after prompt engineering + RAG are exhausted
AI Agents — ReAct, Planning, Tool Use	Agents	LangChain YouTube — LangGraph tutorials	LangGraph docs + AutoGen docs + CrewAI docs	Multi-tool agent: web search + Python REPL + document retrieval + calculator	Agent completes 10-step research tasks with >80%	LangGraph over simple LangChain agents —

Skill / Technology	Label	YouTube Resource	Alt Resource	Practical Project	Stop Condition	Notes
					success rate reliably	stateful, cyclical, more reliable

Phase 4 — Capstone Projects

PROJECT #4: RAG Document Search Engine

- ChatPDF clone with production-grade RAG — multi-document upload, chat, citations
- Advanced RAG stack: chunking strategies (semantic + sliding window) + re-ranking (Cohere) + HyDE query expansion
- Stack: Next.js + FastAPI + OpenAI Embeddings + Qdrant vector DB + GPT-4o
- Evals: Build a 100-question eval dataset; measure recall@5, answer accuracy, latency
- Hybrid search: BM25 + vector search merged with Reciprocal Rank Fusion
- Citations: every answer links to exact document chunks with page numbers
- Deployed: Railway (backend) + Vercel (frontend). Live demo URL in README.
- GitHub: Portfolio Project #4 — RAG is the #1 most requested AI skill in job interviews

PROJECT #5: Multi-Tool AI Research Agent

- Stateful multi-tool research agent built with LangGraph — not simple LangChain chains
- Tools: Tavily web search, URL reader (Jina AI), Python REPL, Wikipedia, Qdrant retrieval, calculator
- React UI showing agent reasoning steps + tool calls in real-time (streaming)
- Human-in-the-loop: approval required for code execution and web browsing
- Reliability: retry logic, fallback models (GPT-4o → Claude → Gemini), graceful degradation
- LangSmith tracing: every agent run fully traced and observable
- Deployed: Fly.io (agent backend) + Vercel (frontend). GitHub: Portfolio Project #5.

PHASE 5 — AI Application Engineering & Production SaaS *— Month 14–17*

This phase is about building real AI products — not demos. You will master the AI application frameworks, implement evals, build AI SaaS with real monetisation, and learn the production patterns that separate amateurs from engineers.

Skill / Technology	Label	YouTube Resource	Alt Resource	Practical Project	Stop Condition	Notes
LangChain LCEL — Chains, Memory, Callbacks	LangChain	LangChain official YouTube channel	LangChain Python + JS docs	Refactor your RAG system to use LangChain Expression Language	Build chains in LCEL without fighting the abstraction — know when to skip it	Know when NOT to use LangChain — sometimes 20 lines of Python is cleaner
LlamaIndex — Enterprise Document RAG	LlamaIndex	LlamaIndex YouTube + llamaindex.ai blog	LlamaIndex docs + LlamaHub connectors	Enterprise knowledge base: 10+ data sources, structured + unstructured	Query across SQL + vector + graph simultaneously; sub-200ms response	Better than LangChain for document-heavy RAG with complex data types
LangGraph — Production Agent Orchestration	LangGraph	LangChain — LangGraph full tutorial (YouTube)	LangGraph docs + LangGraph Cloud docs	Multi-agent customer support system with escalation to human	Agents handle edge cases; no infinite loops; full state persistence	LangGraph is the production standard for stateful AI agents in 2024
Vercel AI SDK — Production Streaming	Vercel AI	Vercel YouTube — AI SDK tutorials	Vercel AI SDK docs (sdk.vercel.ai)	Implement multi-model streaming with fallback and token counting in Next.js	Stream to 1000 concurrent users; first token <300ms; fallback works silently	The best abstraction for AI streaming in Next.js — use it as the default
LLM Evaluation Pipelines	LLM Evals	Hamel Husain — LLM Evals (YouTube talks)	LangSmith docs + Braintrust docs + RAGAS library	Set up automated evals: 50+ test cases run on every PR in CI pipeline	Catch LLM regressions before production; eval latency <5 min in CI	RAGAs for RAG evals, LangSmith for tracing, Braintrust for regression testing
Prompt Versioning & Management	Prompt Mgmt	Hamel Husain — prompt management talks	LangSmith prompt hub + PromptLayer docs	Version control all prompts; A/B test in production with metrics	Know which prompt version is in production; roll back in <60 seconds	Prompts are code — treat them with the same rigor as application code
Model Routing & Cost Optimisation	Model Routing	Simon Willison — LLM cost talks (YouTube)	OpenAI tokenizer + pricing docs + LiteLLM docs	Build intelligent model router: fast/cheap for simple tasks, powerful for complex	Reduce AI API costs by 60% vs always using GPT-4o — measured savings	LiteLLM for unified API; route by task complexity, latency, cost per token
LLM Response Caching — Semantic Cache	LLM Cache	Zep AI — semantic caching (YouTube)	GPTCache docs + Redis vector search docs	Implement semantic cache: exact match + fuzzy similarity for LLM responses	Cache hit rate >40% on typical user sessions; cost reduction measurable	Semantic caching = exact queries and semantically

Skill / Technology	Label	YouTube Resource	Alt Resource	Practical Project	Stop Condition	Notes
						similar ones both cache-hit
Multi-modal AI — Vision, Audio, Documents	Multi-modal	Fireship — GPT-4 Vision (YouTube)	OpenAI Vision + Whisper + GPT-4o docs	Receipt scanner: image → structured expense JSON; voice note → task list	Extract structured data from images with >92% field accuracy	Multi-modal is exploding — vision, audio, video inputs are standard now
AI Product Design — UX Patterns	AI UX	Amelia Wattenberger — AI UX talks (YouTube)	Anthropic + OpenAI blog posts on AI product design	Redesign an AI app based on 5 user sessions — apply AI-specific UX patterns	Users trust and return to the product (Day-7 retention >30%)	Streaming indicators, uncertainty signals, edit/regenerate — unique to AI apps

Phase 5 — Capstone Project

PROJECT #6: Production AI SaaS — Launch & Monetise

- Build AND launch a real AI SaaS product that accepts payments and has real users
- Ideas: AI writing assistant for a niche, AI meeting summariser, AI legal doc analyser, AI code reviewer
- AI Stack: RAG + agents + structured outputs + streaming + multi-modal as needed
- Must have: Clerk auth, Stripe usage-based billing, plan tiers, email notifications, admin dashboard
- Evals: Automated eval pipeline runs on every deploy — regressions blocked from reaching prod
- Observability: LangSmith tracing, Sentry, PostHog, uptime monitoring (Better Uptime)
- Model routing: fast/cheap model for simple tasks, GPT-4o only when needed — cost measured
- Launch: Product Hunt, HackerNews, IndieHackers, Twitter/X — get real paying users
- GitHub: A LIVE product with paying users > any demo project. This is Portfolio Project #6.

PHASE 6 — AI Infrastructure: Self-Hosting, GPU Inference & Scaling *— Month 18–20*

This is the phase that most tutorials completely skip. Understanding how to serve LLMs efficiently — locally with Ollama, in production with vLLM, at scale with Ray Serve — is what separates a junior AI engineer from a senior one. This phase covers GPU inference, quantization, batch processing, and self-hosting strategies.

Section A — Local LLM Hosting & Experimentation

Skill / Technology	Label	YouTube Resource	Alt Resource	Practical Project	Stop Condition	Notes
Ollama — Local LLM Hosting	Ollama	Matt Williams (Ollama creator) — talks (YouTube)	Ollama docs + ollama.ai + Open WebUI docs	Run Llama 3.1, Mistral, Phi-3 locally; build a local AI API for development	Serve local LLM from a REST API; replace OpenAI calls in dev with zero code change	Local LLMs = free development + privacy for sensitive data; Ollama is the tool
Model Quantization — GGUF, GPTQ, AWQ	Quantization	TheBloke — quantization explainers (YouTube)	llama.cpp docs + GPTQ docs + bitsandbytes docs	Quantize Llama 3.1 8B to 4-bit; measure quality vs speed tradeoff	Run 8B model on 8GB VRAM with <5% quality degradation vs full precision	Q4_K_M GGUF = best quality/speed tradeoff for most use cases on consumer GPU
llama.cpp — Efficient CPU/GPU Inference	llama.cpp	Andrej Karpathy — llama.cpp explanation	llama.cpp GitHub README + performance benchmarks	Build a CLI tool using llama.cpp Python bindings for offline document QA	Local inference at 20+ tokens/sec on MacBook Pro; memory usage optimised	Foundation of Ollama — understanding llama.cpp helps debug performance issues
Open-Source Model Ecosystem	OSS Models	Yannic Kilcher — model reviews (YouTube)	LMSYS Chatbot Arena + HuggingFace Open LLM Leaderboard	Evaluate 5 open-source models on your domain task; pick best for cost/quality	Make informed model selection decisions — not just defaulting to GPT-4o	Llama 3.1, Mistral, Phi-3, Qwen2, Gemma — know the landscape

Section B — Production GPU Inference

Skill / Technology	Label	YouTube Resource	Alt Resource	Practical Project	Stop Condition	Notes
vLLM — High-Throughput LLM Serving	vLLM	vLLM team talks (YouTube + vllm.ai blog)	vLLM docs + GitHub + PagedAttention paper	Deploy Llama 3 with vLLM; load test to 100 req/s; measure throughput vs latency	Serve 100+ concurrent LLM requests with GPU utilisation >80%	vLLM's PagedAttention = 24x throughput improvement over naive serving.

Skill / Technology	Label	YouTube Resource	Alt Resource	Practical Project	Stop Condition	Notes
						Industry standard.
Ray + Ray Serve — Distributed AI Serving	Ray Serve	Anyscale — Ray Serve tutorials (YouTube)	Ray docs + Anyscale blog posts	Deploy a scalable model serving pipeline with Ray Serve on a multi-GPU machine	Auto-scale model replicas based on request queue depth; zero-downtime deploys	Ray is what companies use when vLLM alone isn't enough — Uber, Spotify, etc.
Triton Inference Server (intro)	Triton	NVIDIA — Triton intro talks (YouTube)	NVIDIA Triton docs + MLPerf benchmarks	Deploy a HuggingFace model on Triton; compare throughput vs vLLM	Understand when Triton's batching strategy beats vLLM — specific use cases	Triton for non-LLM models (vision, audio) — LLMs usually better on vLLM
GPU Memory Management & Optimization	GPU Opt	Tim Dettmers — quantization talks (YouTube)	bitsandbytes docs + FlashAttention paper	Profile GPU memory usage with torch.profiler; identify and fix memory bottlenecks	Fit a 7B model that previously OOM'd — specific memory optimisation steps taken	KV cache management, FlashAttention, gradient checkpointing — know the tools
Batch Inference Pipelines	Batch Infer	Anyscale — batch inference (YouTube)	vLLM offline inference docs + Ray Data docs	Build a batch pipeline: process 10K documents overnight with vLLM offline mode	Process 10K documents in <2 hours at <$5 on GPU instance vs $50 on OpenAI API	Batch inference = 3–5x cheaper than online inference for non-real-time tasks
Serverless GPU Inference — Modal + Replicate	Serverless GPU	Modal team talks (YouTube + modal.com blog)	Modal docs + Replicate docs + RunPod docs	Deploy a custom model on Modal with cold-start <10s; auto-scale on demand	Model serves requests within budget; no idle GPU costs; cold start acceptable	Modal = best for Python-native AI inference. Replicate = easiest model sharing.
Model Serving Cost Optimisation	Serve Cost	Simon Willison — inference cost talks	Together AI + Fireworks AI + Groq pricing comparison	Compare inference cost: OpenAI vs Together AI vs self-hosted vLLM on GPU	Know exact $ per million tokens across 5 providers; make data-driven decision	Groq (LPU) for speed, Together AI for open models, OpenAI for quality — know when

Section C — Modern Deployment Stack

Deployment Philosophy: Modern AI startups don't start with Kubernetes. They use Vercel + Railway + Fly.io to ship fast. Kubernetes becomes necessary at ~50+ microservices or when you have specific scaling requirements that managed platforms can't meet. Don't add K8s complexity until you genuinely need it.

Skill / Technology	Label	YouTube Resource	Alt Resource	Practical Project	Stop Condition	Notes
Vercel — AI Frontend + Edge Functions	Vercel	Vercel official YouTube channel	Vercel docs + edge runtime docs	Deploy Next.js AI app; use edge functions for low-latency LLM routing	Sub-100ms edge function response globally; Vercel AI SDK streaming working	Default for all Next.js AI frontends — free tier is generous
Railway — Backend + Database Hosting	Railway	Railway official YouTube + blog	Railway docs + railwayapp.io	Deploy FastAPI + Postgres + Redis + Qdrant on Railway with one config file	Full AI stack running on Railway in <30 minutes from zero	Best managed backend for early-stage AI SaaS — simple, affordable, fast
Fly.io — Global Edge Deployment	Fly.io	Fly.io official blog + YouTube	Fly.io docs + flyctl CLI docs	Deploy a stateful AI agent service across 3 regions with fly.io	Requests route to nearest region; failover works; multi-region latency <200ms	Better than Railway for latency-sensitive AI services needing global distribution
Modal — Serverless AI Functions	Modal	Modal official YouTube + blog	Modal docs (modal.com/docs)	Deploy a custom embedding model + image generation pipeline on Modal	Custom model cold-starts in <8s; GPU auto-provisioned on demand	Modal is game-changing for AI engineers — run any Python + GPU workload serverlessly
Replicate — Model API Marketplace	Replicate	Replicate official YouTube	Replicate docs + replicate.com/explore	Deploy an open-source model on Replicate; expose as API; integrate into app	Anyone can call your model via API with one line of code	Great for sharing models; also useful for using community models without hosting
Docker — Containerisation	Docker	TechWorld with Nana — Docker full course (YouTube)	Docker official docs + Play with Docker	Dockerise FastAPI + Next.js with multi-stage builds; push to registry	Any project you build runs in Docker with one command	Foundation for everything — Railway, Fly.io, Modal all use containers internally
AWS Core — EC2, S3, Lambda, ECS	AWS	FreeCodeCamp — AWS Cloud Practitioner course (YouTube)	AWS Skill Builder free tier + AWS docs	Deploy an AI service on ECS Fargate; store model outputs in S3; Lambda for triggers	Understand AWS cost model; set billing alerts; know when EC2 beats Lambda	Focus on EC2, S3, RDS, Lambda, ECS, SageMaker — skip the rest initially
Kubernetes — When & How	K8s	TechWorld with Nana — K8s	Kubernetes docs + k3s for local clusters	Deploy an AI service on a local k3s cluster	Write basic K8s manifests; understand	K8s when: 50+ microservices,

Skill / Technology	Label	YouTube Resource	Alt Resource	Practical Project	Stop Condition	Notes
		complete course (YouTube)		— write Deployment + HPA YAMLs	when to escalate from Railway to K8s	custom networking needs, or specific compliance reqs

Phase 6 — Capstone Project

PROJECT #7: AI Developer Tool (Cursor / Copilot Clone)

- AI code intelligence tool: semantic code search + AI code review + PR comment bot
- Code embeddings: voyage-code-2 (best code embedding model) → Qdrant vector DB
- Self-hosted: local Llama 3.1 via Ollama for development; vLLM in production for cost savings
- GitHub App: PR comment bot that reviews code changes, detects security issues, suggests improvements
- CLI tool: 'ai-review <file>' runs AI code review locally using local Ollama model
- Batch pipeline: nightly job re-indexes entire codebase for fresh semantic search
- Deployed: Modal (inference) + Fly.io (API) + Vercel (dashboard). Open-sourced.
- MLOps: W&B tracks model quality; LangSmith traces every review; evals in CI pipeline
- GitHub: Portfolio Project #7 — developer tools get viral stars and direct job offers

PHASE 7 — Data Engineering for AI Products *— Month 21–22*

AI models are only as good as their data. As an AI Product Engineer you must understand how to build reliable data pipelines, version datasets, monitor data quality, and create feature stores. This phase covers the data engineering skills that most AI tutorials ignore but every production AI system requires.

Skill / Technology	Label	YouTube Resource	Alt Resource	Practical Project	Stop Condition	Notes
ETL Pipelines — Extract, Transform, Load	ETL	Zach Wilson — data engineering YouTube channel	Fundamentals of Data Engineering (Joe Reis, free chapter)	Build an ETL pipeline: scrape 10K web pages → clean → chunk → embed → Qdrant	Pipeline processes 10K documents reliably; failures retry; partial runs resume	ETL is the foundation of all RAG systems — your data quality = your AI quality
Prefect — Modern Pipeline Orchestration	Prefect	Prefect official YouTube channel	Prefect 2.0 docs (prefect.io/docs)	Orchestrate your document ingestion pipeline with Prefect flows + deployments	Pipeline runs on schedule; failures alert Slack; retries with exponential backoff	Prefect > Airflow for new projects — better DX, easier local dev, Python-native
Apache Airflow — Industry Standard Orchestrator	Airflow	Marc Lamberti — Airflow (YouTube, best Airflow channel)	Airflow official docs + Astronomer Academy (free)	Migrate a Prefect pipeline to Airflow DAG; deploy on Astro Cloud free tier	Write idempotent DAGs; understand XCom, sensors, hooks, connections	Airflow is everywhere in enterprise — know it even if you prefer Prefect
Data Ingestion — APIs, Webhooks, CDC	Data Ingestion	Fireship + Zach Wilson — ingestion patterns	Airbyte docs + dbt docs + Debezium docs	Build a multi-source ingestion pipeline: REST APIs + webhooks + DB CDC	New data appears in vector DB within 5 minutes of source update	CDC (change data capture) keeps AI knowledge base fresh — critical pattern
dbt — Data Transformation	dbt	dbt Labs YouTube channel — dbt fundamentals	dbt docs + dbt courses on dbt Learn (free)	Build a dbt project: raw data → staging → marts for an AI analytics dashboard	dbt models are tested, documented, versioned; lineage graph shows dependencies	dbt is the standard for SQL-based data transformation in modern data stacks
Feature Stores — Feast + Tecton (concepts)	Feature Store	Made With ML — feature stores (YouTube)	Feast docs + Hopsworks docs + Feature Store book	Set up Feast locally; define features for a user personalisation AI model	Feature values available online (<10ms) and offline for training — no skew	Prevents training-serving skew — the most common ML production bug
Data Versioning — DVC	Data Version	Iterative.ai — DVC tutorials (YouTube)	DVC docs (dvc.org) + iterative.ai blog	Version a dataset used for RAG; roll back to previous version after quality drop	Reproduce any experiment from 6 months ago — same data, same results	DVC for datasets is what Git is for code — non-negotiable for reproducibility

Skill / Technology	Label	YouTube Resource	Alt Resource	Practical Project	Stop Condition	Notes
Data Quality Monitoring — Great Expectations	Data Quality	Great Expectations official YouTube	Great Expectations docs + Soda Core docs	Add data quality checks to your ETL pipeline — alert on quality failures	Pipeline fails fast on bad data; stakeholders notified before AI quality degrades	Bad data silently degrades AI quality — monitoring catches it before users do
Vector DB Management & Index Maintenance	VectorDB Ops	Qdrant YouTube channel + blog posts	Qdrant production ops docs + Weaviate ops guide	Build a vector DB management dashboard: index health, drift detection, re-indexing	Detect and alert on embedding distribution drift; auto-trigger re-indexing	Embeddings drift as models update — production RAG needs index maintenance

PHASE 8 — MLOps, AI Safety & Reliability Engineering *— Month 23–24*

This final phase covers the operational and safety aspects of production AI systems. AI without proper monitoring degrades silently. AI without security is a liability. These skills are increasingly required at senior AI engineer level and are a massive competitive differentiator.

Section A — MLOps & Model Operations

Skill / Technology	Label	YouTube Resource	Alt Resource	Practical Project	Stop Condition	Notes
MLflow — Experiment Tracking & Model Registry	MLflow	MLflow official YouTube channel	MLflow docs + 'MLOps Engineering at Scale' book	Track all experiments for a fine-tuning job; register and version winning model	Reproduce any experiment from 6 months ago — params, metrics, artifacts all stored	MLflow = open source W&B alternative; most enterprises already have it deployed
Weights & Biases — Full MLOps Platform	W&B	Weights & Biases YouTube (full tutorials)	W&B docs + W&B courses (free at wandb.ai)	Set up W&B for LLM fine-tuning; use W&B Prompts for LLM tracing	Every training run tracked; prompt traces visible; team can compare runs	W&B is used at 90% of AI companies — learn it deeply, not just the basics
LangSmith — LLM Observability	LangSmith	LangChain YouTube — LangSmith tutorials	LangSmith docs + LangChain blog	Instrument your RAG pipeline with LangSmith; set up automated eval suite	Every LLM call traced; evals run automatically; regressions caught in CI	LangSmith is to LLM apps what Datadog is to traditional apps — essential
AI Evaluation Framework Design	AI Evals	Hamel Husain — evaluations (YouTube, key talks)	RAGAS docs + Evals cookbook (OpenAI GitHub)	Design end-to-end eval suite: correctness + faithfulness + relevance + latency	Eval suite catches 90%+ of regressions before they reach production	Evals are the AI equivalent of unit tests — automate them in CI from day one
Model Monitoring — Drift & Degradation	Model Monitor	Evidently AI YouTube channel	Evidently AI docs + Arize AI docs + Fiddler docs	Set up data drift detection for embeddings; alert on output quality degradation	Alert fires within 1 hour of measurable quality degradation — tested with injection	LLM apps drift as prompts change, models update, data shifts — monitor all three

Skill / Technology	Label	YouTube Resource	Alt Resource	Practical Project	Stop Condition	Notes
CI/CD for AI — GitHub Actions + Eval Gates	AI CI/CD	TechWorld with Nana — GitHub Actions (YouTube)	GitHub Actions docs + DVC CI docs	Automated pipeline: test → eval → build → deploy; eval gate blocks bad deploys	No LLM regression reaches production; CI pipeline completes in <10 minutes	Eval gate = minimum passing score required before deployment proceeds

Section B — AI Security & Reliability

Skill / Technology	Label	YouTube Resource	Alt Resource	Practical Project	Stop Condition	Notes
Prompt Injection Defense	Prompt Inject	Simon Willison — prompt injection talks (YouTube)	OWASP LLM Top 10 + Lakera AI blog	Build a prompt injection test suite; implement layered defense in a RAG app	App resists 95%+ of known prompt injection attack patterns — tested	Prompt injection is the #1 AI security risk. Every production AI app is vulnerable.
LLM Guardrails — NeMo + Guardrails AI	Guardrails	NVIDIA NeMo Guardrails (YouTube + blog)	NeMo Guardrails docs + Guardrails AI (guardrails-ai) docs	Add content moderation + topic restriction + PII detection to your AI SaaS	Guardrails block 100% of obvious abuse; false positive rate <2% on legit queries	Use Guardrails AI (open source) for input/output validation — production proven
PII Detection & Data Leakage Prevention	PII / DLP	Microsoft Presidio talks (YouTube)	Microsoft Presidio docs + AWS Comprehend docs	Add PII scrubbing to your document ingestion pipeline before embedding	Zero PII reaches the LLM context or gets stored in logs — verified by test suite	Presidio (open source) or AWS Comprehend — scrub before LLM, scrub after
Hallucination Mitigation Strategies	Hallucination	Eunsol Choi — hallucination research talks	Vectara Hallucination Leaderboard + FactScore paper	Implement RAG + chain-of-verification + citation forcing to reduce hallucination	Hallucination rate <5% on your domain benchmark — measured with FactScore	RAG alone doesn't stop hallucination — add citation forcing + CoVe verification
AI Red-Teaming — Adversarial Testing	Red Team	Anthropic safety team talks (YouTube)	Anthropic red-teaming guide + NIST AI RMF	Red-team your own AI product: attempt jailbreaks, data extraction, DoS	Document all found vulnerabilities; 90%+ mitigated before launch	Structured red-teaming before launch = prevents embarrassing

Skill / Technology	Label	YouTube Resource	Alt Resource	Practical Project	Stop Condition	Notes
						failures at scale
Human-in-the-Loop System Design	HITL	Anthropic + Scale AI — HITL talks	Scale AI docs + Argilla docs + Label Studio docs	Add HITL approval flow to your AI agent for high-risk decisions	High-risk actions never execute without human approval; audit log maintained	HITL is not a weakness — it is a feature. Users trust AI more with a human backstop.
AI Reliability Engineering	AI SRE	Chip Huyen — ML systems reliability talks	Chip Huyen — Designing ML Systems book	Design SLA for your AI product; implement circuit breakers + fallbacks	P99 latency <3s; uptime >99.5%; automatic fallback to cheaper model on errors	Circuit breaker: if GPT-4o fails, auto-route to Claude. Users never see the error.
Responsible AI & Compliance Basics	AI Ethics	Anthropic + DeepMind safety talks (YouTube)	EU AI Act summary + NIST AI RMF + Anthropic AUP	Audit your AI product against EU AI Act high-risk categories	Documented compliance assessment; terms of service accurately describes AI use	EU AI Act enforcement starts 2026 — knowing the basics now = competitive advantage

12-Month Execution Plan

This is your week-by-week battle plan. Each month has a primary focus, deliverable, and career action. Treat this as law — deviate only with good reason.

Month	Primary Focus	Secondary Focus	Project Milestone	Career Action
Mo 1	Advanced JS + TypeScript	Next.js App Router basics	Deploy AI Chat UI v1 — streaming works on Vercel	Update LinkedIn: 'Building AI-powered products'
Mo 2	React internals + Performance	Testing: Vitest + Playwright	AI Chat UI v2 — full test suite, multi-model support	Write first technical article: 'How LLM streaming works in Next.js'
Mo 3	Node.js + Fastify REST APIs	PostgreSQL + Prisma schema	REST API for AI Chat with auth + DB persistence	Apply to 3 full-stack JS roles as backup; use project as portfolio
Mo 4	Redis + Authentication	Background jobs: BullMQ	Full-stack AI SaaS boilerplate — open source on GitHub	Post on HN 'Show HN' — boilerplates get massive traction
Mo 5	Stripe billing + System Design	API security + rate limiting	Add usage-based billing to SaaS boilerplate	Apply to 5 AI startup jobs — use boilerplate as main portfolio piece
Mo 6	Python + FastAPI + async	NumPy + Pandas data processing	Python document parser: PDF/DOCX → JSON via FastAPI	Post Python + AI article. Join LangChain and HuggingFace Discord servers.
Mo 7	File I/O + data pipelines	Prefect intro orchestration	Async document processing pipeline integrated with JS backend	Write a case study post on the full-stack AI pipeline you built
Mo 8	ML fundamentals + Transformers	HuggingFace ecosystem basics	Implement mini GPT following Karpathy's video — understand internals	Attend an AI meetup or virtual AI conference — network actively
Mo 9	LLM APIs + Advanced Prompting	Embeddings + vector databases	Prompt playground with A/B testing; semantic search over 10K docs	Apply to AI Engineer roles — first serious wave of applications
Mo 10	RAG — naive to advanced	Re-ranking + HyDE + RAGAS evals	ChatPDF clone with eval benchmarks — 100-question test suite	Post RAG deep-dive article — this builds massive credibility in AI circles
Mo 11	AI Agents — LangGraph	LangSmith tracing + observability	Multi-tool research agent with full reasoning UI and HITL approval	Apply aggressively — 5 portfolio projects + tracing = very strong profile
Mo 12	Fine-tuning basics	LLM evals in CI pipeline	Fine-tune Llama 3.1 on domain task; add eval gate to all projects	Start targeting Senior AI Engineer roles — you now have a strong portfolio

⍰ GitHub Portfolio Strategy — 7 Career-Defining Projects

Your GitHub is your resume. These 7 projects, built in order, prove you are a top-tier AI Product Engineer. Each is designed to demonstrate a specific high-value skill combination that employers and clients pay premium for.

PROJECT #01: ai-chat-streaming

- Demonstrates: Production Next.js AI UI — streaming, multi-model, TypeScript strict, full test suite
- Key signals: Vercel AI SDK, streaming architecture, Zustand, TanStack Query, Playwright E2E tests
- README must have: Live demo link, architecture diagram, 'How streaming works' technical section
- Where to share: r/webdev, r/ChatGPT, dev.to article, Twitter/X thread with a demo GIF
- Target: 200+ GitHub stars — frontend devs + AI curious devs will both engage

PROJECT #02: ai-saas-starter

- Demonstrates: Full production backend — auth, multi-tenancy, usage billing, queues, observability
- Key signals: Fastify, Prisma, Redis, Clerk, Stripe, BullMQ, Sentry, PostHog, Docker Compose
- README must have: One-command local setup, architecture diagram, how to extend for your AI use case
- Where to share: HackerNews 'Show HN' post — free SaaS boilerplates consistently get 500+ upvotes
- Target: 500+ GitHub stars — this is pure utility; developers will bookmark and return

PROJECT #03: ai-document-intelligence

- Demonstrates: Python AI data engineering — document parsing, async pipelines, structured extraction
- Key signals: FastAPI, PyMuPDF, python-docx, asyncio, OpenAI structured outputs, Pydantic, Prefect
- README must have: Supported formats table, benchmark numbers (documents/minute), API docs link
- Where to share: IndieHackers, HuggingFace forums, Python subreddits
- Target: 150+ stars — niche but valuable; document AI is a huge enterprise market

PROJECT #04: production-rag-engine

- Demonstrates: Advanced RAG — the #1 most-demanded AI skill in every job description
- Key signals: Qdrant, hybrid search, re-ranking, HyDE, RAGAS evals, LangSmith tracing, benchmarks

- README must have: Benchmark results (recall@5, answer accuracy), comparison vs naive RAG, architecture diagram
- Where to share: LangChain Discord, AI Twitter, HuggingFace forums, LinkedIn article
- Target: 400+ stars — RAG projects consistently go viral in AI communities

PROJECT #05: ai-research-agent

- Demonstrates: LangGraph agents — stateful, multi-tool, human-in-the-loop, production reliability
- Key signals: LangGraph, Tavily, HITL approval, real-time reasoning UI, fallback models, LangSmith
- README must have: Demo video/GIF of agent solving complex task, tool architecture diagram
- Where to share: AI Twitter, r/LocalLLaMA, LinkedIn — agents are the hottest AI topic
- Target: 600+ stars — agent engineering is cutting-edge; very few devs can build reliable ones

PROJECT #06: [your-ai-saas-product]

- Demonstrates: Full AI product ownership — ideation, build, launch, monetisation, retention
- Key signals: LIVE product, paying users, production infrastructure, real metrics (DAU, MRR, churn)
- README must have: Product link, MRR if comfortable sharing, tech stack, lessons learned
- This is NOT a demo — it must be a live product users can sign up for and pay for
- Signal: A ₹10K MRR AI SaaS = instant Senior Engineer offers. Product sense + engineering is the rarest combo.

PROJECT #07: ai-code-intelligence-tool

- Demonstrates: AI developer tooling + self-hosted LLMs + vLLM + Modal serverless GPU
- Key signals: voyage-code-2 embeddings, Qdrant, local Llama 3.1 via Ollama, vLLM prod serving, GitHub App
- README must have: GitHub App install instructions, architecture diagram, self-hosting guide
- Where to share: Dev.to, HackerNews, GitHub Explore — developer tools get massive organic distribution
- Target: 1000+ stars — developer tools go viral; Cursor/Copilot alternatives are extremely popular

⍰ 12 Critical Mistakes — AI Transition Traps to Avoid

Read this section every 3 months and audit yourself honestly against each mistake.

#1 — Tutorial Hell	Watching 50 hours of tutorials without shipping. Cure: build something broken within 7 days of starting any topic. Broken projects teach more than perfect tutorials.
#2 — Skipping Backend	Jumping from frontend directly to AI/LLM APIs. AI products need auth, queues, rate limiting, billing, webhooks. Phase 2 is non-negotiable — do not skip it.
#3 — Theory Before Practice	Spending months on linear algebra and statistics before building anything. You need ML intuition for building products. The maths can come later if you pursue research.
#4 — One LLM Dependency	Hard-coding to GPT-4o. Build with a provider abstraction from day one. Providers go down, raise prices, and get outcompeted. OpenAI going down should not take your product down.
#5 — Zero Evals Strategy	Shipping LLM features with no automated testing. LLM outputs are non-deterministic. Evals are AI's equivalent of unit tests. Set up RAGAS/LangSmith from day one.
#6 — Ignoring AI Costs	Building AI apps with no awareness of per-query cost. At scale, a $0.05/query cost with 10K users = $500/day = $180K/year. Model routing + caching is existential.
#7 — Private GitHub	Keeping all projects private until they are 'perfect'. Ship half-finished projects public. Your GitHub activity graph and project history matter as much as your code quality.
#8 — No AI Security	Shipping AI products with no prompt injection defense or output validation. The OWASP LLM Top 10 exists for a reason. Every AI product in production will be attacked.
#9 — Generic AI Engineer Identity	Being a generic 'AI engineer' with no niche. 'AI engineer specialising in developer tools' or 'AI engineer for legal-tech' is 10x more findable and 2x more compensated.
#10 — Skipping Data Engineering	Building RAG systems without understanding data pipelines. Your AI is only as good as your data ingestion. Bad data silently destroys AI quality — no error message tells you.
#11 — No Monitoring After Deploy	Shipping and forgetting. LLM apps drift silently — prompts change, models update, data shifts. Without monitoring, you discover issues from angry users, not dashboards.
#12 — Skipping Local LLMs	Only knowing how to call OpenAI API. Knowing how to run local models (Ollama, vLLM) saves money in development, enables privacy-sensitive use cases, and makes you dramatically more hireable.

© 2024 AI Product Engineer Roadmap | Build. Ship. Learn. Repeat.

⍰ Skills Resistant to AI Automation Until 2040

AI will automate boilerplate. Here are the skills that remain valuable because they require judgment, systems thinking, domain knowledge, and human context.

AI Systems Architecture	Choosing WHICH AI approach for a given problem — RAG vs fine-tuning vs agents vs prompting — is judgment AI cannot make. Designing AI systems is a human-judgment meta-skill.
LLM Evaluation & Quality Engineering	Building evals, benchmarks, quality systems for AI outputs. As AI generates more, humans who define and measure 'good enough' become more valuable, not less.
AI Security & Red-Teaming	Securing AI systems against prompt injection, jailbreaks, data leakage, adversarial inputs. EU AI Act enforcement creates massive regulatory demand for AI security specialists.
Data Engineering for AI	Building reliable data pipelines, feature stores, data quality systems. Garbage in, garbage out — data engineering is the least glamorous but most business-critical AI skill.
AI Infrastructure & Cost Engineering	Optimising LLM inference costs, managing GPU infrastructure, building batch pipelines. Companies spend millions on AI compute — engineers who cut costs by 60% are irreplaceable.
Product Engineering Judgment	Deciding what to build, for whom, why. Connecting AI capabilities to genuine user problems. Product sense + AI engineering is the rarest and most compensated combination.
Domain-Specific AI Engineering	Deep expertise in one vertical (legal AI, medical AI, developer tools, fintech AI). Domain knowledge + AI engineering cannot be replicated by AI for that domain.
Human-AI Collaboration Design	Designing systems where humans and AI collaborate effectively — where to put humans in the loop, how to build trust, how to handle AI errors at scale. Fundamentally human-judgment work.
AI Teaching & Technical Communication	Explaining AI systems clearly to non-technical stakeholders. As AI permeates every industry, engineers who can translate technical reality to business decisions are extremely valuable.
Multi-modal & Emerging AI Integration	Consistently integrating the newest AI capabilities (vision, audio, video, code) into products before competitors. Being a fast adapter to new AI APIs is a durable career moat.

⍰ Realistic Salary Progression (India + Global Remote)

Now (Year 0)	₹6L–10L | Frontend Developer. Use this time to build — not job hop.
Year 1–2 (Phase 1+2 complete)	₹10L–18L | Full-Stack Engineer with AI integration skills. Target: product startups building AI features.
Year 2–3 (Phase 3+4 complete)	₹18L–28L | AI Engineer. Target: AI-first startups, big tech AI teams, consultancies with AI practice.
Year 3–4 (Phase 5+6 complete)	₹28L–45L | Senior AI Product Engineer. Target: Series B+ startups, MNCs, remote global roles ($60K+).
Year 4–5 (Phase 7+8 complete)	₹45L–70L | Lead/Staff AI Engineer or Founding Engineer. Remote global: $90K–$130K USD.
Year 5–6 (full mastery)	₹70L–1Cr+ | Principal Engineer, AI Architect, CTO of AI startup, or independent AI consultant.
Accelerator: Launch a SaaS	A live AI SaaS with ₹50K MRR adds ₹10L–25L to your negotiating power immediately.
Accelerator: Remote global	Target US/EU remote roles. Indian AI engineers in massive demand. $100K–$150K is realistic by Year 5.
Accelerator: AI infra niche	vLLM + Ray Serve + GPU inference specialists earn 50% premium over general AI engineers — very undersupplied.
Accelerator: AI security niche	Prompt injection + guardrails + red-teaming = emerging niche with regulatory tailwind. Premium pricing guaranteed.

⍰ Essential Reading List

You Don't Know JS (6 books)	Kyle Simpson — GitHub free. Read all 6 before finishing Phase 1. Fills every JS gap.
Designing ML Systems	Chip Huyen — best practical book on building real ML products. Read during Phase 4–5.
Build a LLM from Scratch	Sebastian Raschka — implement GPT from scratch. Understand LLMs at the code level.
Designing Data-Intensive Applications	Martin Kleppmann — system design bible. Read during Phase 2–3. Essential for senior roles.
Fundamentals of Data Engineering	Joe Reis & Matt Housley — read during Phase 7. Best introduction to data engineering for AI.

The Mom Test	Rob Fitzpatrick — validate AI product ideas with real users. Essential before building Phase 5 SaaS.

Final Principle: The difference between a top 1% AI engineer and an average one is NOT knowledge — it is EXECUTION VELOCITY. How fast do you go from idea to shipped? Build something every week. Your GitHub tells the story of your growth. Start building today — not when you feel 'ready'.